

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent progress in deep learning optimization has yielded a family of algorithms—among them Schedule-Free optimizers, AdEMAMix, MARS, and Lion—that, despite their disparate motivations, share a defining structural feature: each reworks the classical momentum mechanism. In a largely independent theoretical line of work, provable acceleration of stochastic gradient descent (SGD) in the noise-dominated regime has been obtained by decoupling the momentum coefficient from the weight assigned to the current gradient. This paper establishes explicit and, to our knowledge, previously unrecognized algebraic correspondences between these two lines of work, demonstrating that each of these recent optimizers can be recovered—through an exact change of variables applied to its canonical update rule—as an accelerated-SGD update composed with weight averaging, preconditioning, or a coordinate-wise sign operation. This unifying view is not merely taxonomic: it converts the accelerated-SGD principle into concrete, falsifiable predictions about the regimes in which each method should help. We test these predictions in a controlled study of a 150M-parameter decoder-only transformer trained on C4 language modeling in the noise-dominated small-batch regime. AdEMAMix, whose update most closely realizes accelerated SGD, attains the strongest performance among the methods we compare, and the predicted advantages recede—exactly as the theory anticipates—once training leaves the noise-dominated regime at large batch sizes. Building on this analysis, we introduce Simplified-AdEMAMix, which matches the performance of AdEMAMix across both the small- and large-batch settings while dispensing entirely with its second momentum buffer and the attendant memory and scheduling overhead. Our code will be released upon acceptance.

1 INTRODUCTION

Driven by the escalating cost of large-scale pre-training, the recent literature on deep learning optimization has produced a succession of new algorithms, notably Lion (Chen et al., 2023), Schedule-Free SGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these optimizers were advanced from distinct motivations and are typically presented as independent designs, they converge on a single structural theme: each reworks the momentum scheme that governs the parameter update. This shared characteristic, largely unexamined in the literature, is the point of departure for our analysis.

A separate and largely self-contained body of theoretical research has addressed the acceleration of gradient descent under stochastic noise. Classical momentum methods, such as heavy-ball or Nesterov momentum, suffice to accelerate deterministic gradient descent—particularly on quadratic objectives—yet provably fail to accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). This limitation motivated a sequence of alternative momentum schemes engineered to recover provable acceleration in the presence of gradient noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). The unifying observation we draw from this literature is structural: every accelerated SGD method it proposes can be expressed as a single mechanism—decoupling the momentum coefficient from the weight assigned to the current gradient in the parameter update. We formalize this observation as a canonical update template and, in Appendix B, verify by explicit change of variables that all four methods we consider reduce to it.

Our central contribution is to bridge these two research directions through an explicit and exact algebraic correspondence between their update rules. We prove that the Schedule-Free SGD iterate is mathematically equivalent to an accelerated SGD update followed by weight averaging, and we show that optimizers such as Lion, Schedule-Free AdamW, and AdEMAMix are characterized exactly as the composition of an accelerated SGD update with a pre-

conditioner or a coordinate-wise sign operation. These correspondences are derivations at the level of the update recursions rather than new convergence guarantees; we obtain each by reducing a published update rule to a common canonical form, working with the canonical version of each method and noting where practical implementations add bias correction, clipping, or scheduling. Although fragments of these relationships have been noted before (Defazio, 2021), the correspondence between this recent generation of optimizers and accelerated SGD has, to the best of our knowledge, never been established in full. Making the connection precise reframes a set of seemingly disparate heuristics as instances of a single accelerated-SGD principle and, crucially, yields concrete and falsifiable predictions about the regimes in which each method should—and should not—help.

We validate these predictions in a controlled study of a 150M-parameter decoder-only transformer, trained on 15B tokens of C4 at a small batch size of 32k tokens so that optimization proceeds firmly in the noise-dominated regime that the accelerated-SGD account concerns. The empirical picture aligns with the theory on each point it predicts. Schedule-Free AdamW tracks the accelerated SGD-based AdamW optimizer of Algorithm 3—precisely the behavior their proven equivalence requires—and the accelerated methods yield a consistent, if modest, improvement in this regime. Equally in keeping with the account, these advantages recede once the batch size grows large enough that the noise-dominated regime no longer holds. Because the analysis delineates both where these optimizers should win and where they should cease to, the large-batch behavior functions as a genuine test of the framework rather than a mere corroboration; we return to this point when we diagnose the large-batch failure of Schedule-Free AdamW.

Our main contributions are the following:

1. **A unifying algebraic correspondence.** We establish exact update-rule connections between accelerated SGD and the recently proposed optimizers Schedule-Free SGD, AdEMAMix, MARS, and Lion, showing by explicit change of variables that each canonical update is an accelerated SGD step composed with weight averaging, preconditioning, or a coordinate-wise sign operation.
2. **Empirical confirmation in the noise-dominated regime.** In a controlled comparison on a 150M-parameter decoder-only transformer, evaluated by validation loss under a common training protocol, AdEMAMix—the method whose update most closely realizes accelerated SGD—attains the lowest loss, corroborating the central qualitative prediction of the analysis.
3. **A regime boundary predicted by the theory.** Consistent with the equivalence to accelerated SGD, the advantage of these methods over Adam disappears at large batch sizes; concretely, we show that Adam equipped with a momentum schedule already matches AdEMAMix once training leaves the noise-dominated regime.
4. **A mechanistic diagnosis of Schedule-Free AdamW at scale.** At large batch sizes Schedule-Free AdamW falls well behind AdamW with cosine decay, and we attribute this gap—ruling out the competing order-of-operations explanation via a direct LProp comparison—to the intrinsic coupling between the momentum and weight-averaging coefficients built into Schedule-Free optimizers.
5. **Simplified-AdEMAMix.** Guided directly by the analysis, we introduce Simplified-AdEMAMix, which reinstates an explicit, tunable weight on the current gradient and, in our experiments, retains the performance of AdEMAMix across both the small- and large-batch regimes while eliminating the second momentum buffer together with its memory footprint and scheduling overhead.

2 RELATED WORK

We situate our contribution at the confluence of two strands of prior work: the theory of accelerated SGD variants, and the practical optimizers whose update structure, as we will show, instantiates that theory.

The theoretical strand originates with Jain et al. (2018), who introduced an accelerated SGD variant with provably improved convergence rates on the least-squares problem. Kidambi et al. (2018) subsequently simplified its update rule and formally established that classical momentum affords no acceleration in this setting, sharpening the case for a decoupled momentum scheme. A line of follow-up work (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended these guarantees to general convex and strongly convex objectives under a range of assumptions, so that the decoupling principle is now supported across a broad class of problems.

A parallel strand of practical optimizers has repeatedly, if often implicitly, rediscovered the same structure. Lucas et al. (2019) formed the update from a weighted combination of several momentum terms carrying distinct coefficients,

while Ma & Yarats (2019) designed an optimizer grounded explicitly in the framework of Jain et al. (2018). More recently, Chen et al. (2023) obtained an optimizer through genetic search that, like the earlier accelerated SGD variants, assigns separate weights to the current gradient and to momentum, and Pagliardini et al. (2024) proposed blending two momentum time scales in the final update. That such independently motivated designs keep converging on the same decoupled structure is precisely the regularity our analysis explains, and it is what allows us to treat these methods within one common framework rather than as a catalogue of unrelated heuristics.

3 BACKGROUND

3.1 MOMENTUM

Momentum is a canonical technique for accelerating the convergence of gradient descent in the deterministic setting. For weights w_t and momentum coefficient β , the momentum update reads:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a standard variance-reduction technique in stochastic optimization that suppresses the noise carried by individual iterates. Rather than returning the final iterate w_T , one maintains a weighted running average $\bar{w}_T$ of the iterates, governed by averaging weights γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout this paper, every instance of weight averaging employs coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We now introduce the general framework that subsumes the accelerated SGD methods discussed above and serves as the canonical template against which we compare the existing optimizers:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \tag{1}$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are (possibly time-dependent) scalar coefficients and g_t is the stochastic gradient evaluated at w_t ; the subscript ‘a’ marks the coefficients that obey this accelerated SGD formulation. The defining feature of Equation (1) is that the current gradient enters the update through two distinct channels—once, with memory, through the momentum buffer m_t , and once, directly and without memory, through the term $\alpha_{a,t} g_t$ —so that its effective weight is decoupled from the momentum coefficient.

Two observations frame the analysis that follows. First, setting $\alpha_{a,t} = 0$ severs the direct channel and collapses the update to standard SGD with momentum, recovering the classical scheme as a special case. Second, the accelerated SGD algorithms proposed in the literature—including those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023)—all fall within this single framework; we make each such equivalence explicit, by an individual change of variables, in Appendix B.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

In this section we establish exact connections between accelerated SGD and existing optimizers, encompassing the Schedule-Free family, Lion, MARS, and AdEMAMix. These derivations are not merely interpretive analogies: each

reduces a published update rule—in its canonical form, setting aside implementation-level details such as bias correction and gradient clipping, which we note where they arise—to the canonical form of Equation (1) through an explicit change of variables. Building on the understanding these reductions yield, we then propose a simplified variant of AdEMAMix that retains a single momentum term yet, in our experiments, matches AdEMAMix across both the small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a recently introduced constant-learning-rate optimizer that dispenses with an explicit learning-rate schedule while remaining competitive with well-tuned schedules. Adopting the notation of Defazio et al. (2024), its update equations read:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here y_t denotes the current model weights, at which the gradient is evaluated, whereas x_t denotes the averaged weights returned at evaluation time.

Our strategy is to re-express these dynamics in terms of y_t together with an auxiliary momentum variable m_t , defined as:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Expanding this definition and simplifying, we obtain a recursion for m_t :

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus m_t obeys exactly the momentum recursion of Equation (1) with $\beta_{a,t} = 1 - c_t$. Given this identification, we derive the corresponding update for y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Hence y_t obeys the weight update of Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$, $\alpha_{a,t} = \gamma(1 - \beta)$, and $w_t = y_t$. The iterate y_t of Schedule-Free SGD therefore instantiates the accelerated SGD framework exactly. It remains to account for x_t , the iterate actually returned at evaluation, whose dynamics we now characterize:

$$\begin{aligned}
x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\
x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\
x_{t+1} (1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\
x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}.
\end{aligned}$$

Thus x_t is a weighted average of the y_t iterates, and unrolling the recursion shows that, for constant c_t , it is precisely an exponential moving average of y_t . Taken together, these two derivations establish our first main result: *Schedule-Free SGD is exactly accelerated SGD followed by weight averaging*.

This decomposition renders the two sources of Schedule-Free SGD’s effectiveness explicit:

1. improved performance relative to standard SGD with momentum, inherited directly from its equivalence to accelerated SGD; and
2. the ability to train at a constant learning rate without any schedule, enabled by weight averaging (specifically tailed weight averaging; see Section 4.1.3).

The construction further confers two practical advantages specific to Schedule-Free SGD/Adam:

- it requires no additional memory for weight averaging; and
- it eliminates the weight-averaging coefficient as a separate hyperparameter.

This economy, however, is not without cost: because a single set of coefficients governs both momentum and averaging, the two cannot be tuned independently. In Section 5.1 we show that precisely this coupling is what prevents Schedule-Free SGD from scaling gracefully to large batch sizes, turning an apparent convenience into a concrete liability outside the noise-dominated regime.

Examining the limiting and default values of β reveals how this single equivalence recovers several familiar regimes as special cases.

4.1.1 CASE: $\beta = 0.0$

As already observed by Defazio et al. (2024), setting $\beta = 0$ reduces Schedule-Free SGD to plain SGD with weight averaging; because $c_t = 1/t$, averaging is applied from the very first step.

4.1.2 CASE: $\beta = 1.0$

As also noted by Defazio et al. (2024), setting $\beta = 1$ reduces the method to standard momentum SGD, with the momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

At the default value $\beta = 0.9$, our decomposition pins down all three coefficients simultaneously:

- since c_t scales as $1/t$, the momentum coefficient grows as $1 - 1/t$;
- the ratio of the weight on the current gradient to the weight on momentum is held fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$; and
- weight averaging effectively spans the most recent 10% of the iterates.

This third property is what we term *tailed* weight averaging, and it is the mechanism at the center of the large-batch analysis of Section 5.1.

4.2 LION

The same lens applies directly to Lion (Chen et al., 2023), whose update rule is:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Because the two momentum buffers m'_t and m_t employ distinct coefficients β_1 and β_2 , the argument of the sign operation is again a linear combination in which the current gradient and the momentum term carry decoupled weights—exactly the signature of Equation (1). Lion (Chen et al., 2023) is therefore precisely an accelerated SGD update composed with a coordinate-wise sign operation.

4.3 MARS

We now show that the practical variant of the recently proposed optimizer MARS (Yuan et al., 2024), denoted MARS-Approx, likewise instantiates the accelerated SGD framework, here composed with a preconditioning step. Setting aside bias correction and clipping—implementation details that leave the momentum structure intact—its update equations are:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t are the first- and second-moment estimates, respectively, and x_t denotes the model parameters. Introducing the change of variables $\hat{m}_t = m_t - \gamma g_t$ recasts the update in the equivalent form:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

In this form the momentum recursion for $\hat{m}_t$, together with the direct γg_t term in the parameter update, is manifestly an instance of the accelerated SGD framework. The analysis thus locates the distinguishing feature of MARS not in its momentum—which coincides with accelerated SGD—but in its preconditioner, which differs from that of AdamW. We evaluate the empirical consequences of this design choice in Section 5.

4.4 ADEMAMix

The recently proposed AdEMAMix optimizer (Pagliardini et al., 2024) bears a close structural resemblance to accelerated SGD-based AdamW. Where accelerated SGD forms a linear combination of the current gradient and a single momentum term, AdEMAMix maintains two momentum buffers with different coefficients and combines them; the

full procedure is stated in Algorithm 1. As we now show, this apparent difference is superficial: under a natural parameter choice, AdEMAMix realizes the accelerated SGD template directly.

Consider the variant with $\beta_1 = 0$. Pagliardini et al. (2024) report that this variant is nearly indistinguishable in performance from the full method at small batch sizes, and our own experiments in Section 5 confirm this. With $\beta_1 = 0$, AdEMAMix coincides with the general accelerated SGD framework of Equation (1). This structural match is only part of the correspondence; we further show that the schedules AdEMAMix prescribes for the momentum coefficient β_3 and for α —the coefficient governing the relative weight on the current gradient—closely track the schedules that theory prescribes for accelerated SGD (Gupta et al., 2023).

Specifically, in smooth convex optimization, achieving stochastic acceleration requires a momentum schedule of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximates precisely this schedule through its prescribed warmup of β_3 toward its target value.

A second correspondence concerns the manner in which the two schemes accumulate past gradients. In accelerated SGD, momentum is kept in the standard (theory-style) form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas the momentum update in Algorithm 1 takes the EMA form:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients to m_t in accelerated SGD grows in proportion to t ; the coefficient α in AdEMAMix is scaled up in exactly the same proportion. Because AdEMAMix reproduces both the momentum schedule and this linear growth of the past-gradient weight, it is the closest practical realization of accelerated SGD among the optimizers we study—which, on the accelerated-SGD account, is precisely why it attains the strongest performance in Figure 1.

This same alignment also predicts, in advance, where AdEMAMix should struggle. At large batch sizes the variant with $\beta_1 = 0.0$ degrades, as reported by Pagliardini et al. (2024). The explanation follows directly from the theory: Gupta et al. (2023) show that as the batch size grows the weight on the current gradient must be reduced, yet AdEMAMix holds this weight fixed at 1. The accelerated-SGD correspondence therefore pinpoints the fixed unit gradient weight as the specific mechanism responsible for the large-batch degradation, rather than leaving it as an unexplained empirical observation.

This diagnosis furnishes a constructive prescription and motivates the next section, in which we introduce a simplified variant of AdEMAMix that reinstates an explicit, tunable weight on the current gradient, removes the need to schedule α , and retains only a single momentum term. We show empirically that this variant matches AdEMAMix across both the small- and large-batch regimes.

4.5 SIMPLIFIED ADEMAMIX

The preceding analysis translates directly into a concrete design. We propose Simplified-AdEMAMix, an optimizer that discards the second momentum buffer of AdEMAMix and removes the schedule on α altogether, stated formally in Algorithm 2. Two design choices follow from the theory rather than from tuning: momentum is maintained in the theory-style form of Equation (1) rather than as an exponential moving average (EMA), and the current gradient enters the final update through a fixed, tunable weight α —precisely the decoupled gradient weight that the accelerated-SGD account identifies as essential and that fixed-weight AdEMAMix lacks. Setting $\alpha = 0$ recovers standard Adam, up to the appropriate reparameterization of η and β_1 , so that Simplified-AdEMAMix strictly generalizes Adam through a single additional coefficient while shedding, relative to AdEMAMix, one momentum buffer and one schedule. In Section 5 we show that this single-momentum optimizer matches the performance of AdEMAMix across both the

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

- 1: Sample batch B_t .
- 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
- 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
- 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v}} + \epsilon}$
- 5: $w \leftarrow w - \eta N$
- 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
- 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
- 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$

small- and large-batch regimes, so that the removed machinery is empirically dispensable rather than merely redundant in principle.

5 EXPERIMENTS

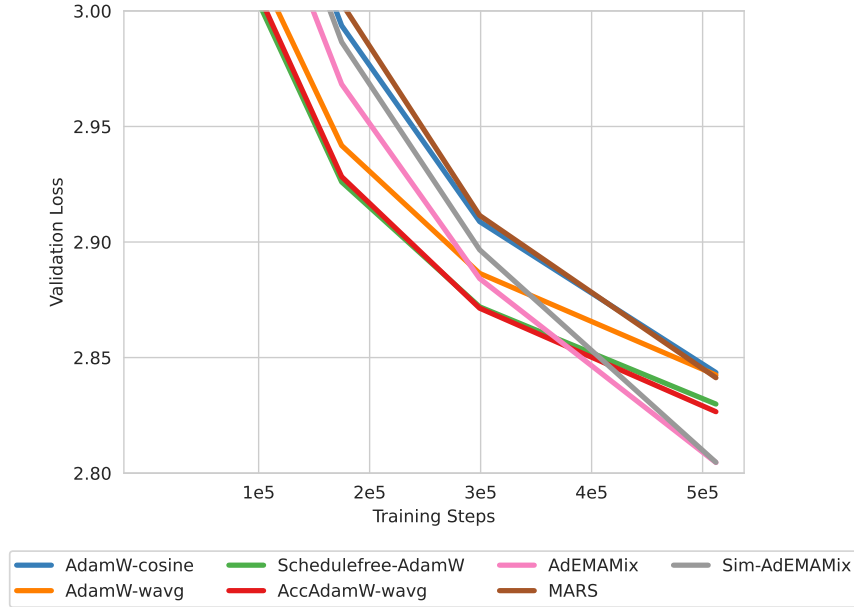


Figure 1: Validation loss of the best run of each optimizer listed in Section 5, for language modeling on a 150M decoder-only transformer trained in the small-batch regime (32k tokens, 15B tokens total). The ordering is exactly the one the accelerated-SGD analysis predicts: AdEMAMix and Simplified-AdEMAMix, the closest practical realizations of accelerated SGD, attain the lowest validation loss; Schedule-Free AdamW and Accelerated AdamW with weight averaging track one another closely, as their proven equivalence requires; and both improve on AdamW with cosine decay. The margins between the leading methods are modest, consistent with the qualitative nature of the prediction.

We now subject the predictions of our analysis to empirical test. All experiments in this section use a 150M-parameter decoder-only transformer trained for language modeling on the C4 dataset, with a sequence length of 1024 and a batch size of 32 sequences, over 15 billion tokens ($\approx 5 \times$ the Chinchilla-optimal token budget). This configuration places training firmly in the noise-dominated regime—the precise setting in which the accelerated-SGD account predicts a measurable advantage for the corresponding optimizers—and thereby isolates the regime that the theory concerns.

Holding this configuration fixed, we compare the following optimization algorithms under a common training protocol:

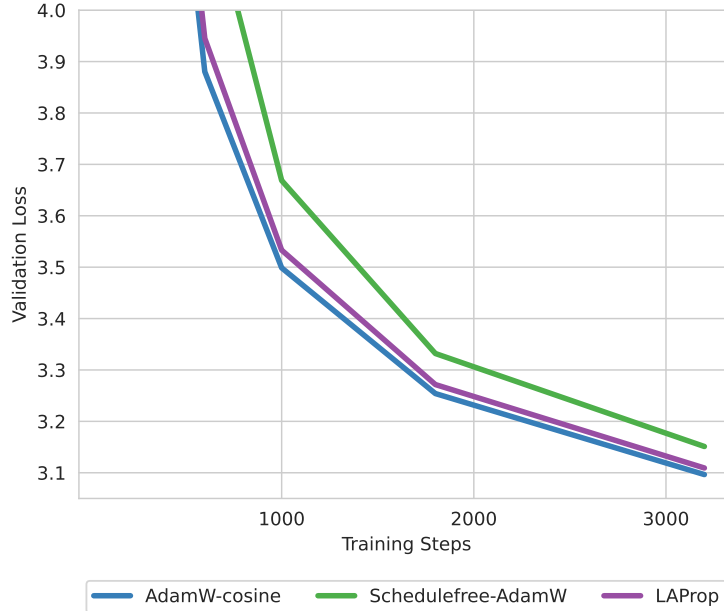


Figure 2: Best runs of AdamW with cosine decay, Schedule-Free AdamW, and LAProp at the large batch size (1m tokens, 3B tokens total). Schedule-Free AdamW trails AdamW markedly, whereas LAProp—which, like Schedule-Free AdamW, applies preconditioning before momentum—remains comparable to AdamW. This contrast isolates the coupling of momentum and weight averaging, rather than the order of momentum and preconditioning, as the source of the gap. Experimental details appear in Section 5.1.

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

Each optimizer is tuned over the hyperparameter sweeps detailed in Appendix A, and, following common practice for this kind of comparison, we report the best run of each; the sweeps are of comparable breadth across methods, so the comparison does not systematically favor any one optimizer.

The results in Figure 1 confirm the analysis on every qualitative point it predicts. Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform comparably—exactly as their proven equivalence requires—and both outperform AdamW with cosine decay and AdamW with tailed weight averaging. AdEMAMix and Simplified-AdEMAMix attain the lowest validation loss overall, which the accelerated-SGD account attributes to their being the closest practical realizations of accelerated SGD. The margins among the leading methods are modest, as befits a qualitative regime prediction rather than a claim of large practical gains; the salient finding is the consistency of the ordering with the theory. That Simplified-AdEMAMix matches AdEMAMix here using only a single momentum term already anticipates the large-batch results of the next section.

5.1 LARGE BATCH SIZE EXPERIMENTS

The small-batch experiments above operate in the regime where the accelerated-SGD account predicts a benefit. To delineate the boundary of that benefit—and thereby to test the analysis where it makes its most falsifiable predictions—

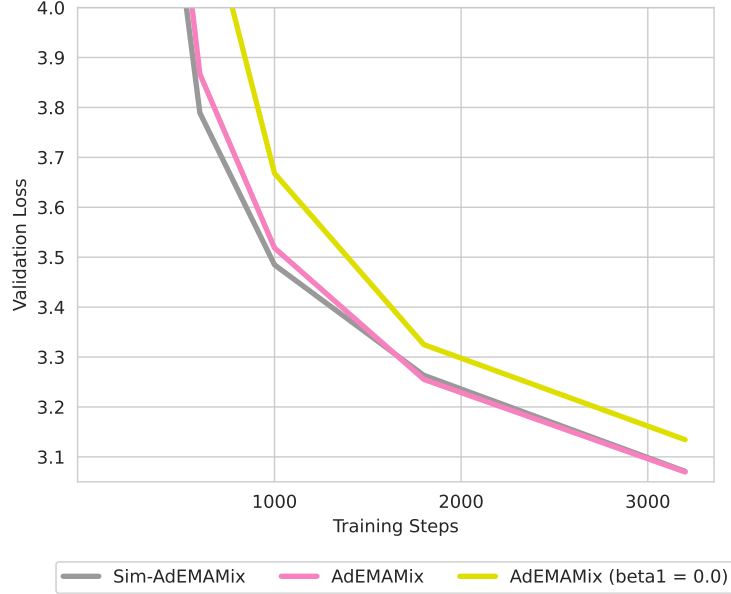


Figure 3: Best runs of AdEMAMix (with and without $\beta_1 = 0.0$) and our Simplified-AdEMAMix at the large batch size (1m tokens, 3B tokens total). Fixing $\beta_1 = 0.0$ in AdEMAMix incurs a clear loss, exactly as the accelerated-SGD account predicts for its fixed unit gradient weight, whereas Simplified-AdEMAMix matches full AdEMAMix using a single momentum term—and does so at $\alpha = 0.0$, i.e., as Adam with a momentum schedule. Experimental details appear in Section 5.1.

we now turn to the large-batch regime, training with a batch size of 1 million tokens over 3 billion tokens ($\approx$ Chinchilla scale). Here the theory makes two predictions: that acceleration should offer little advantage, and, more sharply, that Schedule-Free AdamW should suffer specifically because of the coefficient coupling we identified in Section 4.1.3. Both are borne out, and it is the second, mechanism-specific prediction that most directly corroborates the framework.

Schedule-Free AdamW. As Figure 2 shows, Schedule-Free AdamW falls substantially behind AdamW at the large batch size. We attribute this gap to the coupling between the weight-averaging and momentum coefficients established in Section 4.1.3. At large batch sizes the optimal momentum value lies well below $1 - 1/t$; one can compensate with a faster scaling $\approx 1 - r/t$ for some $r \geq 1$, but a larger r necessarily contracts the effective weight-averaging window, so momentum and averaging cannot both be set well at once. A competing explanation attributes the gap instead to the order of operations: AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning first, rendering it algorithmically akin to LARProp (Ziyin et al., 2021). We rule this alternative out directly—Figure 2 shows AdamW and LARProp performing comparably despite sharing that ordering—which isolates the coefficient coupling, and not the order of operations, as the operative cause of the degradation.

AdEMAMix. Consistent with Pagliardini et al. (2024), Figure 3 shows that fixing $\beta_1 = 0.0$ in AdEMAMix causes a marked drop relative to retaining two momentum terms. The accelerated-SGD analysis supplies the mechanism: the theory of Gupta et al. (2023) requires the weight on the current gradient to diminish as the batch size grows, yet AdEMAMix fixes this weight at 1. Simplified-AdEMAMix reinstates a tunable gradient weight and, as Figure 3 shows, recovers the full performance of AdEMAMix while using only a single momentum term. It does so, moreover, at $\alpha = 0.0$, so that in the large-batch regime Simplified-AdEMAMix reduces to Adam equipped with a momentum schedule—the concrete conclusion that plain Adam, given a momentum schedule, already matches AdEMAMix once training leaves the noise-dominated regime.

6 CONCLUSION

This paper establishes explicit and exact algebraic correspondences between accelerated SGD variants and a recent generation of optimizers—Schedule-Free optimizers, AdEMAMix, MARS, and Lion—showing that the canonical update of each is an accelerated SGD step composed with weight averaging, preconditioning, or a coordinate-wise sign operation. These correspondences are more than taxonomic: they issue falsifiable predictions that our controlled experiments confirm, namely that AdEMAMix, the closest practical realization of accelerated SGD, attains the lowest validation loss in the noise-dominated small-batch regime, and that this advantage recedes—by the specific mechanism the analysis identifies—as the batch size grows.

The same analysis yields a concrete algorithm rather than only an interpretation. Simplified-AdEMAMix dispenses with the second momentum buffer of AdEMAMix, together with its memory footprint and the schedule on α , yet matches AdEMAMix across both the small- and large-batch regimes in our experiments. Taken together, these results indicate that the recent proliferation of momentum-based optimizers is most productively understood through a single accelerated-SGD principle, which clarifies both when such methods help and how they may be simplified without measurable loss of performance. Our study is confined to a single 150M-parameter language model, and confirming that these correspondences and their regime predictions transfer to larger models, longer horizons, and other modalities is a natural and important direction for future work.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.

- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

For reproducibility and to document that the tuning budget is comparable across methods, we report in full the hyperparameter sweeps used for every optimizer. The sweeps for the small-batch experiments are listed first, followed by those for the large-batch experiments.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999 respectively, coinciding with the defaults. Whereas larger batch sizes commonly favor $\beta_2 = .95$, the advantage of a higher β_2 at smaller batch sizes accords with the observations of Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$; that is, we fix β_1, β_2 to the optimal values identified in the preceding sweep. This configuration underperformed the 51.2k-step warmup.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The hyperparameter sweeps for the large batch experiments are provided below:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

This appendix substantiates the claim that the accelerated SGD framework of Equation (1) is genuinely general rather than a convenient special case: for each of the methods of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023), we exhibit an explicit change of variables that reduces its update to this single canonical form.

B.1 AGNES

The update of Gupta et al. (2023) is:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n and the objective is evaluated at x_n . Eliminating v_n in favor of the rescaled variable $-v_{n+1}/\rho_n$, these equations can be rewritten as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

which places x'_{n+1} in the form of Equation (1).

B.2 ASGD

The update of Jain et al. (2018) is:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} and the objective is evaluated at the tail-averaged iterate x . Taking $y_{j-1} - v_j$ (suitably rescaled) as the momentum variable, these equations can be rewritten as

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

which are again exactly of the form of Equation (1).

B.3 MASS

The update of Liu & Belkin (2020) is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t and the objective is evaluated at w_t . Taking the rescaled difference $w_t - u_t$ as the momentum variable, these equations can be rewritten as

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

which are again of the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update of Vaswani et al. (2019) is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

Taking the difference $v_k - w_k$ as the momentum variable, these equations can be rewritten as

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$

which are once more of the form of Equation (1), completing the demonstration that all four methods instantiate the single framework of Equation (1).